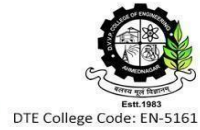




॥ न हि ज्ञानेन सदृशं पवित्रमिह विद्यते ॥
Dr. Vitthalrao Vikhe Patil Foundation's
Dr. Vitthalrao Vikhe Patil
College of Engineering Ahmednagar



E-Commerce

Report On

“Online Book Shopping System”

SUBMITTED BY:

Aniruddha Avinash Pande

(Roll no 108)

Department: Information Technology

Academic Year: 2025-2026

Ecommerce Guide:

Dr.Mrs.P.S.Gayke



॥ न हि ज्ञानेन सदृशं पवित्रमिह विद्यते ॥
Dr. Vithalrao Vikhe Patil Foundation's

Dr. Vithalrao Vikhe Patil
College of Engineering Ahmednagar



DTE College Code: EN-5161

CERTIFICATE

This is to certify that the A Ecommerce Report entitled “**Online Book Shopping System**” is presented by Aniruddha Avinash Pande of S.E. Information Technology Engineering, Pune, under the SPPU, Pune and his work is satisfactory. This work is done during year 2025-2026.

Date:

Guide(IT)
Prof.Dr.Mrs.P.S.Gayke

HOD(IT)
Dr.D.A.Vidhate

ABSTRACT

The **Online Book Shopping System** is a web-based application designed to provide users with a convenient and efficient platform for browsing and purchasing books online. The system allows users to explore a variety of books categorized into different genres such as fiction, non-fiction, self-help, and mystery. Users can view book details including title, author, and price, and add selected books to a virtual shopping cart.

The application features dynamic cart management, where users can add or remove items and view the total cost in real time. A user-friendly interface with interactive elements such as product detail popups and a slide-in cart panel enhances the overall user experience. The system also includes a basic checkout functionality, which currently displays a confirmation message, indicating scope for future integration of payment gateways.

This project is developed using web technologies such as HTML, CSS, and JavaScript, focusing on front-end design and functionality. The Online Book Shopping System demonstrates the fundamental concepts of e-commerce applications, including product listing, cart operations, and user interaction, making it a useful model for understanding real-world online shopping systems.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported and guided me throughout the development of my mini project, “**Online Book Shopping System.**”

I am deeply thankful to our respected Principal, **Dr. R.R. Navthar**, and Head of the Department, **Dr. D.A. Vidhate (IT)**, for providing the necessary facilities and academic environment to complete this project successfully.

I would like to extend my heartfelt thanks to my project guide, **Dr.Mrs.P.S.Gayke** , for his valuable guidance, continuous encouragement, and support throughout the project. His suggestions and insights played an important role in shaping this project.

I also express my gratitude to my friends and classmates for their cooperation, support, and helpful discussions during the development and testing of this system.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this project. This project has been a great learning experience and has helped me enhance my technical and practical skills.

TABLE OF CONTENTS

Sr no.	CONTENTS	PAGE NO
1	Objective	6
2	Introduction	7
3	Methodology/Code	8
4	System Requirements	17
5	Outputs	17
6	References	20
7.	Conclusion	21

1. Objectives

- To develop a user-friendly web application for browsing and purchasing books online
- To develop a user-friendly web application for browsing and purchasing books online
- To provide categorized book listings such as fiction, non-fiction, self-help, and mystery
- To display detailed information about each book including title, author, and price
- To implement an efficient “Add to Cart” functionality for selecting books
- To enable users to view, update, and remove items from the shopping cart
- To calculate and display the total price of selected books dynamically
- To design an interactive and responsive user interface for better user experience
- To simulate the basic functionality of an e-commerce checkout process
- To enhance understanding of front-end web development technologies like HTML, CSS, and JavaScript
- To gain practical experience in building real-world web-based applications

2. Introduction

The **Online Book Shopping System** is a web-based application developed to provide users with a simple, fast, and convenient way to purchase books online. With the rapid growth of internet usage and digital technologies, online shopping has become an essential part of modern life. This system aims to replicate the functionality of an e-commerce website specifically designed for book sales.

The application allows users to browse a collection of books categorized into different genres such as fiction, non-fiction, self-help, and mystery. Each book is displayed with essential details including title, author, and price, helping users make informed purchasing decisions. The system also provides an interactive “Add to Cart” feature, enabling users to select multiple books and manage their shopping cart efficiently.

Additionally, the project includes features like a dynamic cart panel, real-time total price calculation, and a product detail popup for better user interaction. Although the checkout functionality is currently in a basic stage, it demonstrates the working concept of an online purchase system.

This project is developed using front-end technologies such as HTML, CSS, and JavaScript, focusing on creating an attractive and user-friendly interface. The Online Book Shopping System serves as a foundational model for understanding the core concepts of e-commerce applications, including product management, cart operations, and user interaction.

3. Methodology/code

▪ HTML

❖ Navigation And Search Header

This section serves as the primary interface for branding and user actions, containing the logo, a dynamic search bar, and the shopping cart trigger.

- Code :

```
<nav>
  <a class="logo" onclick="render('all')">Folio.</a>
  <input type="text" id="sch" placeholder="Search books..."
  oninput="search(this.value)">
  <button class="btn" onclick="toggleCart(1)"> Cart (<span
  id="count">0</span></button>
</nav>
```

❖ Dynamic Content Containers

These empty div and aside elements act as "slots" where JavaScript injects the book cards, category filters, and cart items

- Code :

```
<div class="cat-nav" id="cats"></div> <div class="main"><div id="grid"
  class="grid"></div></div> <aside id="cart" class="sidebar">
  <div id="cart-list"></div> <div id="totals">
  </div>
</aside>
```

❖ Hero & Informational Section

This provides the visual "hook" of the website and is programmed to hide when a specific category is selected to save space.

- Code :

```
<div id="hero" class="hero">
  <h1>Discover Your Next Story</h1>
  <p>Curated masterworks for the literary soul.</p>
</div>
```


❖ Feedback & Interaction Layers

These components are hidden by default and only appear to provide detailed book information (Modal) or brief action confirmations (Toast)

- Code :

```
<div id="modal" class="overlay" onclick="closeM(event)">
  <div class="modal" id="modal-content"></div>
</div>
<div id="tst" class="toast"></div>
```

▪ JAVASCRIPT

❖ Data Model

This snippet defines the book repository using an array of objects to store metadata like titles, authors, and prices.

- Code :

```
const BOOKS = [
  {id:1, t:"One Hundred Years of Solitude", a:"Gabriel Márquez",
    c:"fiction", p:16.99, bg:"#8B5E3C"},
  {id:4, t:"Atomic Habits", a:"James Clear", c:"self-help",
    p:18.99, bg:"#D4A853"}
]; [cite: 74, 77, 80]
```

❖ Dynamic Filtering & Rendering

The render function filters books based on the selected category and injects the HTML into the page grid.

- Code :

```
function render(cat='all') {
  const filtered = cat==='all' ? BOOKS : BOOKS.filter(b => b.c ===
    cat); [cite: 85]
  document.getElementById('grid').innerHTML = filtered.map(b => `
    <div class="card" onclick="view(${b.id})">
      <div class="cover" style="background:${b.bg}">${b.t}</div>
    </div>`).join(''); [cite: 86, 87, 88, 95]
}
```

❖ State-Based Shopping Cart

This logic manages the user's selection by either incrementing the quantity of an existing item or adding a new one to the cart array.

- Code :

```
function add(id) {  
  const b = BOOKS.find(x => x.id === id); [cite: 116]  
  const item = cart.find(i => i.id === id); [cite: 117]  
  item ? item.q++ : cart.push({...b, q:1}); [cite: 118]  
  updateCart(); [cite: 120]  
}
```

❖ Real-time Calculation & UI Updates

The updateCart function uses the reduce method to calculate the total price and total item count for the user interface.

- Code :

```
function updateCart() {  
  const count = cart.reduce((a,b)=>a+b.q, 0); [cite: 123]  
  const sub = cart.reduce((a,b)=>a+(b.p*b.q), 0); [cite: 124]  
  document.getElementById('count').innerText = count; [cite: 125]  
  document.getElementById('totals').innerHTML =  
    '<b>${sub.toFixed(2)}</b>'; [cite: 131]  
}
```

▪ PHP – Authentication

❖ User Registration & Security

The system validates incoming data and uses the **Bcrypt** algorithm to hash passwords before saving them to the database.

- Code :

```
if ($action === 'register') {  
  $hash = password_hash($data['password'], PASSWORD_BCRYPT);  
  // Secure hashing [cite: 4, 11]  
  db()->prepare("INSERT INTO users (name, email,
```

```

password_hash) VALUES (?, ?, ?)");
    ->execute([$data['name'], $data['email'], $hash]); //
Database insertion [cite: 12]
    jsonResponse(['success' => true]); [cite: 14]
}

```

❖ Login & Cart Synchronization

The login process verifies the hashed password. Upon success, it merges any existing "guest" cart items into the user's permanent account.

- Code :

```

if ($action === 'login') {
    $user = db()->prepare("SELECT * FROM users WHERE email = ?");
    // Fetch user [cite: 22]
    if ($user && password_verify($data['password'],
    $user['password_hash'])) { // Verify credentials
        $_SESSION['user'] = $user; // Create session [cite: 30]

        if (isset($_SESSION['cart_id'])) { // Sync guest cart to
        user [cite: 36]
            db()->prepare("UPDATE carts SET user_id = ?, session_id
            = NULL WHERE id = ?")
            ->execute([$user['id'], $_SESSION['cart_id']]);
            [cite: 44]
        }
    }
}
}

```

❖ Session Control

These endpoints manage the active state of the user, allowing for session identification or complete termination.

- Code :

```

if ($action === 'logout') {
    session_destroy(); // End session [cite: 58]
}

```

```

if ($action === 'me') {
    jsonResponse(['user' => currentUser()]); // Check login status
[cite: 63]
}

```

▪ PHP - BOOK

❖ Book Listing & Filtering

This section handles complex queries to filter books by category, search terms, and special tags (featured, bestsellers, or new arrivals) while implementing pagination for performance.

- Code :

```

if ($action === 'list') {
    $where = ['b.is_active = 1'];
    if ($category) { $where[] = 'c.slug = ?'; $params[] =
$category; }
    if ($search) {
        $where[] = 'MATCH(b.title, b.description) AGAINST(? IN
BOOLEAN MODE)';
        $params[] = '*' . $search . '*';
    }

    $sql = "SELECT b.*, a.name AS author_name, c.name AS
category_name
        FROM books b
        JOIN authors a ON a.id = b.author_id
        JOIN categories c ON c.id = b.category_id
        WHERE " . implode(' AND ', $where) . "
        LIMIT ? OFFSET ?"; // Handles dynamic SQL generation
}

```

▪ PHP- CART

❖ Inventory-Safe Additions

Verifies product availability before updating the cart. It uses ON DUPLICATE KEY UPDATE to efficiently handle repeat items.

- Code :

```
if ($action === 'add') {
    $book = db()->prepare("SELECT stock FROM books WHERE id = ?")-
    >execute([$bookId]);
    if ($book['stock'] < $qty) jsonResponse(['error' =>
    'Insufficient stock'], 409);

    db()->prepare("INSERT INTO cart_items (cart_id, book_id,
    quantity) VALUES (?, ?, ?)
    ON DUPLICATE KEY UPDATE quantity = quantity +
    VALUES(quantity)")
    ->execute([$cartId, $bookId, $qty]); // Update or Insert
}
```

❖ Coupon & Discount Processing

Checks promotional codes for validity, expiration dates, and usage limits before applying them to the user session.

- Code :

```
if ($action === 'apply_coupon') {
    $stmt = db()->prepare("SELECT * FROM coupons WHERE code = ?
    AND is_active = 1
    AND (expires_at IS NULL OR expires_at
    > NOW())");
    $stmt->execute([$code]);
    $coupon = $stmt->fetch();

    if ($coupon) $_SESSION['coupon'] = $coupon; // Store
    discount in session
}
```

❖ Backend: Secure Auth & Search

The backend provides secure entry points for users and advanced data filtering.

- **Authentication:** Uses password_hash for registration and password_verify for secure logins.
- **Search & Filter:** Uses dynamic SQL with MATCH...AGAINST for full-text book searches.

- Code :

```
// auth.php: Secure User Registration
$hash = password_hash($password, PASSWORD_BCRYPT);
db()->prepare("INSERT INTO users (name, email, password_hash)
VALUES (?, ?, ?)")
->execute([$name, $email, $hash]);

// books.php: Dynamic List Retrieval
$sql = "SELECT b.*, a.name AS author_name FROM books b
        JOIN authors a ON a.id = b.author_id WHERE b.is_active =
        1";
```

❖ Transactions: Cart & Orders

This module handles inventory checks and ensures atomic transactions during checkout.

- Code :

```
// orders.php: Atomic Checkout Transaction
$pdo->beginTransaction();
try {
    // 1. Create Order 2. Record Items 3. Decrement Stock
    $pdo->prepare("INSERT INTO orders (...) VALUES (...)")-
    >execute([...]);
    $pdo->prepare("UPDATE books SET stock = stock - ? WHERE id =
    ?")->execute([...]);
    $pdo->prepare("DELETE FROM cart_items WHERE cart_id = ?")-
    >execute([$cartId]);
    $pdo->commit();
} catch (\Throwable $e) { $pdo->rollBack(); }
```

▪ DATABASE

❖ Core Entity Tables

The schema defines the primary objects of the system, including users with role-based access and a categorized book repository linked to authors.

- Code :

-- Users: Stores credentials and roles (customer/admin)

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(120) NOT NULL,  
    email VARCHAR(180) NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    role ENUM('customer','admin') DEFAULT 'customer'  
);
```

-- Books: Detailed metadata including stock levels and ratings

```
CREATE TABLE books (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    author_id INT,  
    category_id INT,  
    price DECIMAL(10,2),  
    stock INT DEFAULT 0,  
    rating DECIMAL(3,2),  
    FOREIGN KEY (author_id) REFERENCES authors(id),  
    FOREIGN KEY (category_id) REFERENCES categories(id)  
);
```

❖ Transactional & Interaction Tables

These tables handle the "active" parts of the site, such as the persistent shopping carts and user-submitted reviews.

- Code :

-- Cart Items: Links books to specific user/session carts

```
CREATE TABLE cart_items (  
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```

        cart_id INT,
        book_id INT,
        quantity INT DEFAULT 1,
        UNIQUE KEY idx_cart_book (cart_id, book_id) -- Prevents
duplicate rows
);

-- Reviews: Stores user ratings and feedback
CREATE TABLE reviews (
    id INT AUTO_INCREMENT PRIMARY KEY,
    book_id INT,
    user_id INT,
    rating INT CHECK (rating BETWEEN 1 AND 5),
    body TEXT,
    is_approved BOOLEAN DEFAULT 0
);

```

❖ Order History & Management

This structure records finalized transactions, capturing a "snapshot" of prices and shipping details at the time of purchase to ensure historical accuracy.

- Code :

```

-- Orders: Master record of a transaction
CREATE TABLE orders (
    id INT AUTO_INCREMENT PRIMARY KEY,
    order_number VARCHAR(20) UNIQUE,
    user_id INT,
    total DECIMAL(10,2),
    status ENUM('pending','confirmed','shipped','delivered'),
    placed_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

```


4. System Requirements

❖ Hardware Requirements

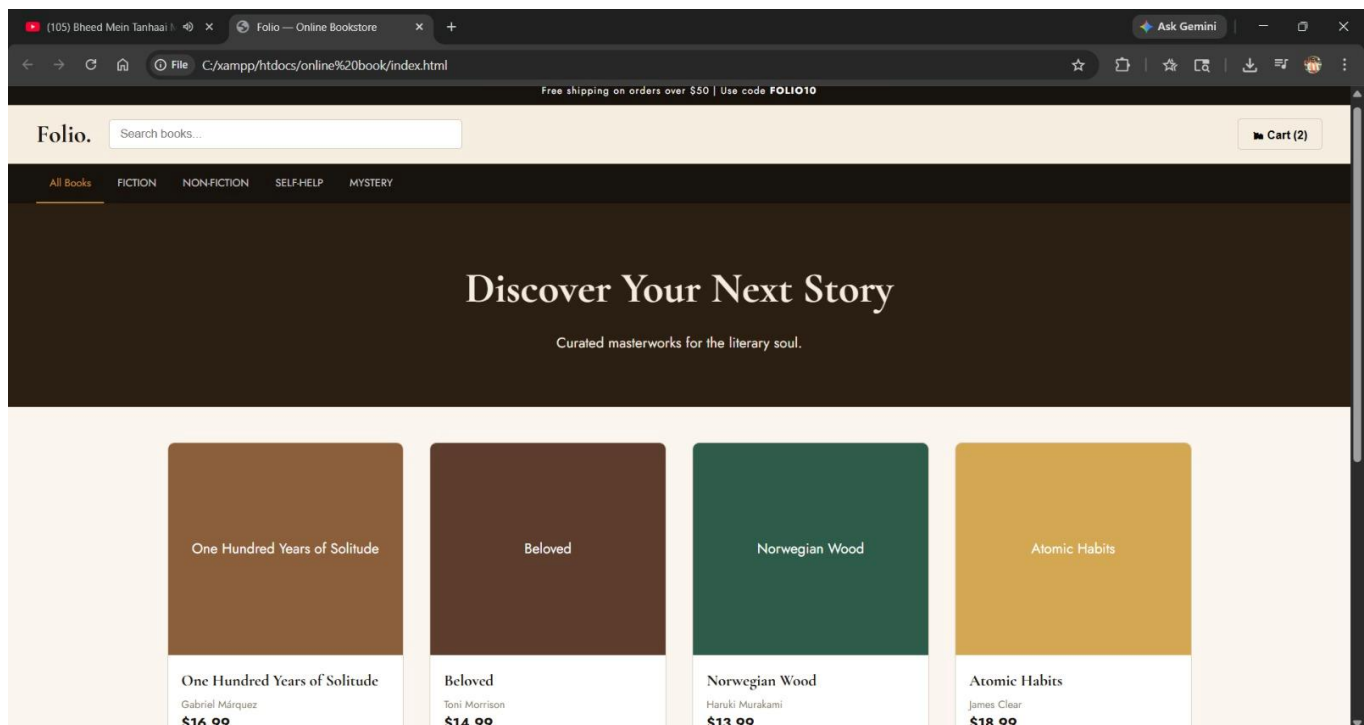
- Computer/Laptop
- Minimum 4GB RAM
- Internet Connection

❖ Software Requirements

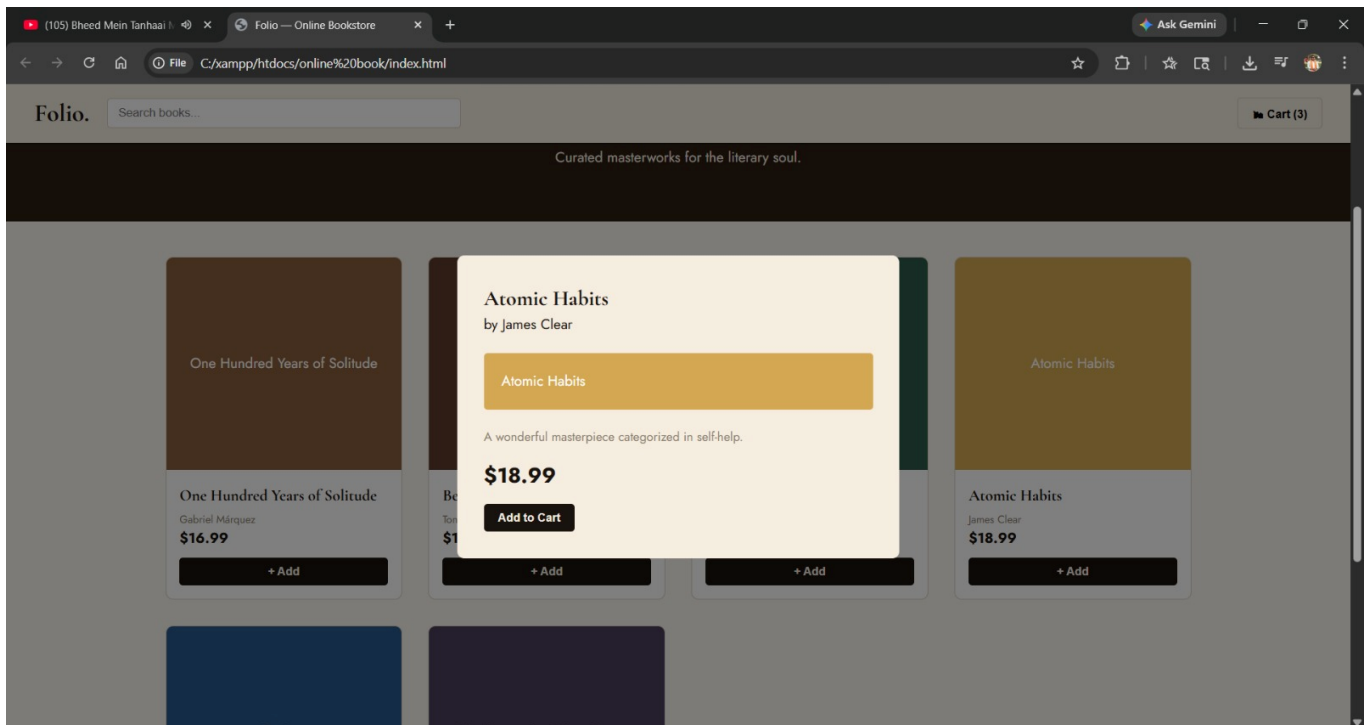
- Web Browser (Chrome, Edge, Firefox)
- XAMPP (optional if backend is used)
- HTML, CSS, JavaScript

5. Outputs

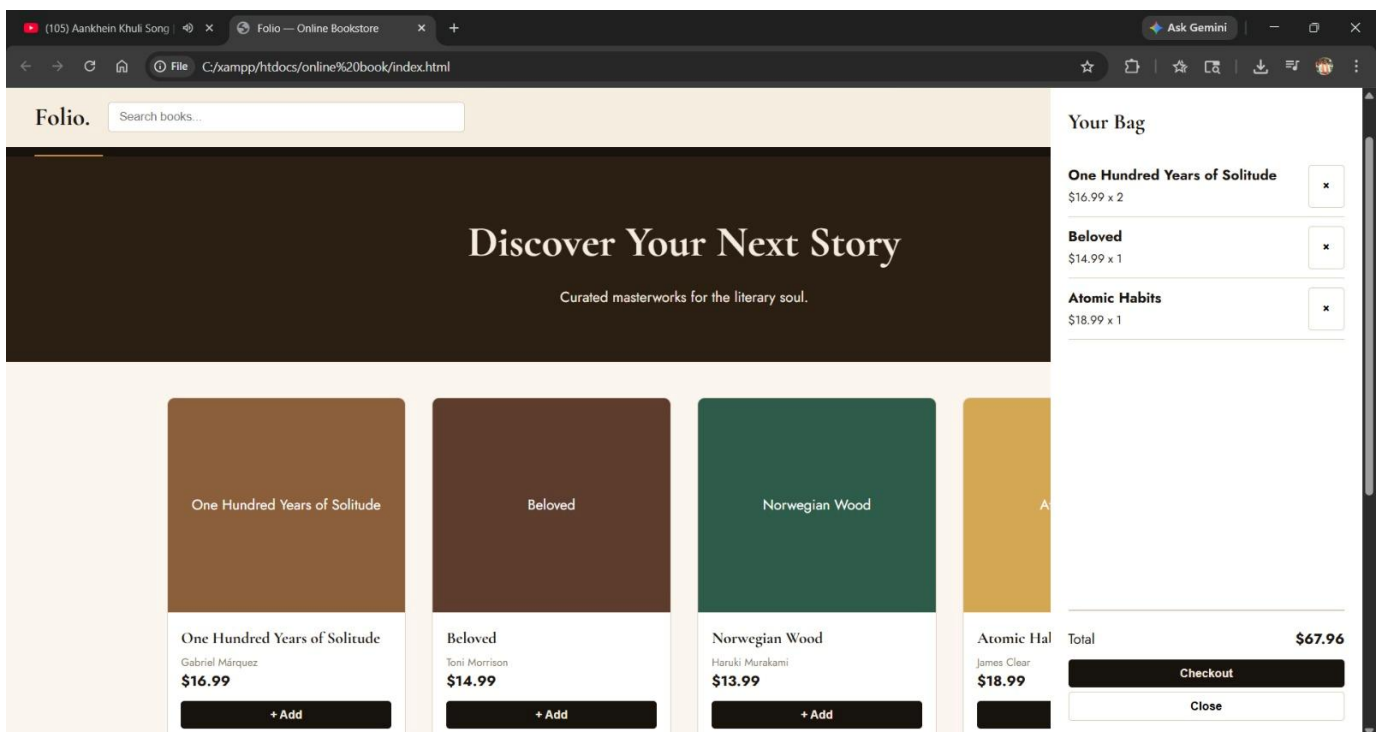
- **Home Page :**



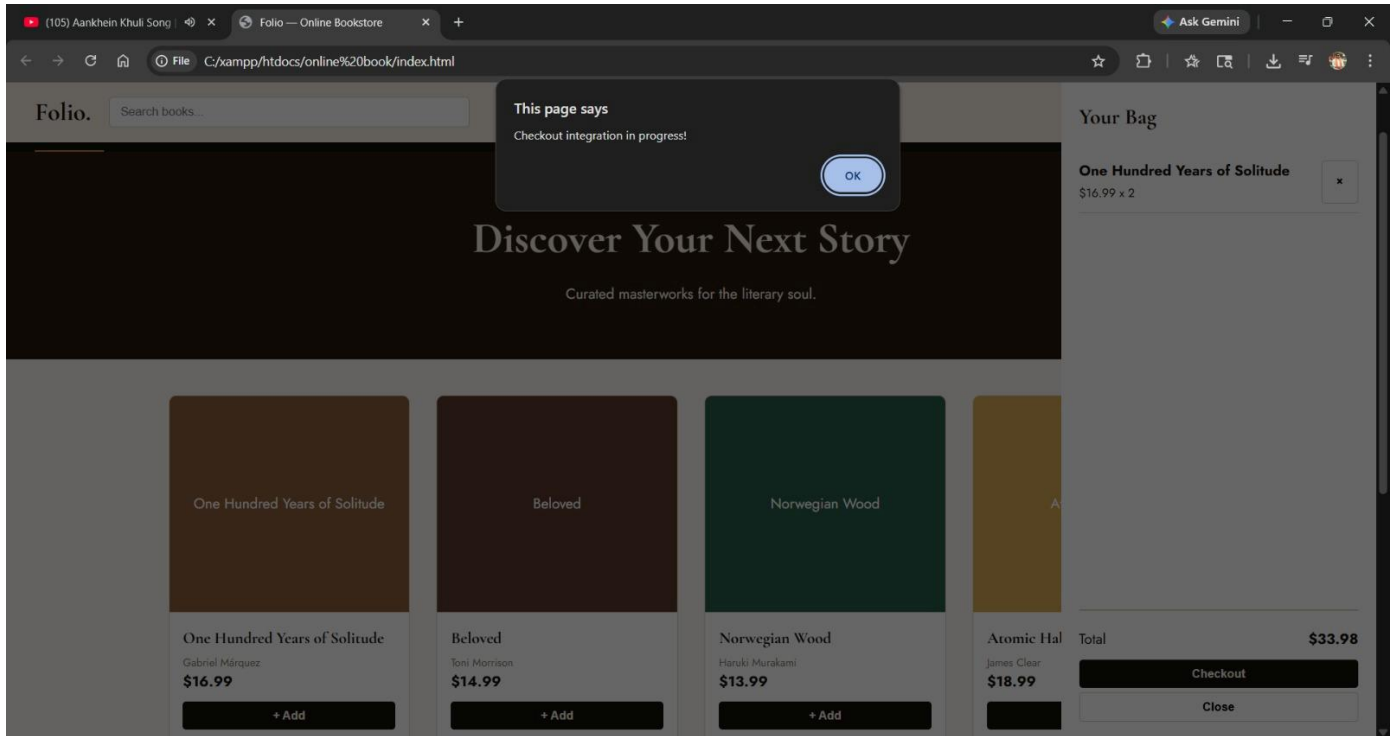
- **Cart Page :**



- **Checkout Page :**



- **Order Process Page :**



6. References

1. **W3Schools**

Website: <https://www.w3schools.com/>

Used for learning HTML, CSS, and JavaScript concepts.

2. **MDN Web Docs**

Website: <https://developer.mozilla.org/>

Used for detailed explanations of JavaScript functions and web APIs.

3. **GeeksforGeeks**

Website: <https://www.geeksforgeeks.org/>

Used for programming concepts and problem-solving techniques.

4. **Stack Overflow**

Website: <https://stackoverflow.com/>

Used for resolving coding issues and debugging.

5. **YouTube**

Website: <https://www.youtube.com/>

Used for tutorials on web development and UI design.

6. **Visual Studio Code**

Used as the development environment for writing and testing code.

7. **Google Chrome**

Used for running and testing the web application.

7. Conclusion

The **Online Book Shopping System** project successfully demonstrates the basic working of an e-commerce web application for purchasing books online. The system provides a simple and user-friendly interface that allows users to browse books, view their details, add them to a cart, and manage their selections efficiently.

The implementation of features such as dynamic cart updates, real-time price calculation, and interactive user interface elements enhances the overall user experience. Although the project currently includes a basic checkout process, it effectively illustrates the core concepts of online shopping systems.

Through the development of this project, significant practical knowledge of web technologies such as HTML, CSS, and JavaScript was gained. It also helped in understanding important aspects of application design, user interaction, and front-end development.

In conclusion, this project serves as a strong foundation for developing more advanced e-commerce applications, and it can be further improved by integrating features like user authentication, payment gateways, and database connectivity in the future.

